

Programming for Essential Digital Skills, Chapter 11

Christoph Walsh

2026

Object Oriented Programming

- Python is what is called an object oriented programming language.
- This means that “objects” in Python represent both data and methods that operate on that data.
- Let’s take a very simple example:

```
a = 1.5
```

- When we do this we are not only binding the value 1.5 to *a*.
- We are also creating a floating-point object which comes with several methods.
- For example: `a.is_integer()`
- In Python, a `class` is an object constructor. It is a blueprint for creating objects.
- By writing a `class` of our own, we can create objects that have a specific structure and methods.

Defining a Class

- We will use a running example of a bank account to illustrate this concept.
- At its simplest level, a bank account has an account holder and a balance.
- We can create a simple class of a bank account as follows:

```
class BankAccount:  
  
    def __init__(self, account_holder, opening_balance=0):  
        self.account_holder = account_holder  
        self.balance = opening_balance
```

Initialization Function

- When defining a class, we always define one function inside called `__init__()`. This initializes an object.
- The `__init__` method runs automatically whenever an object of a certain class is created.
- The parameter `self` refers to the particular object being created.
- The expressions `self.account_holder` and `self.balance` are the object's attributes.

Creating Objects

Let's create two objects with the class BankAccount:

```
account1 = BankAccount("Ailill", 1005)  
account2 = BankAccount("Medb", 1000)
```

We can see these have the class BankAccount:

```
type(account1)
```

```
<class '__main__.BankAccount'>
```

Accessing Attributes

We can access the name of the account holder for account 1 with:

```
account1.account_holder
```

```
'Ailill'
```

and their balance with:

```
account1.balance
```

```
1005
```

Adjusting Attributes

We can adjust attributes of an object directly with:

```
account2.balance = 1100  
account2.balance
```

1100

Adding Methods to a Class

```
class BankAccount:

    def __init__(self, account_holder, opening_balance=0):
        self.account_holder = account_holder
        self.balance = opening_balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print("Insufficient funds")
```

Using Methods

```
account1 = BankAccount("Ailill", 1005)
account1.deposit(100)
account1.balance
```

1105

- Notice that we didn't need to provide something for `self` as an argument to `deposit`.
- The object we are applying the method to passes everything in `self` to the method.

Using Methods

- Let's try withdraw 600 twice from Medb's bank account:

```
account2 = BankAccount("Medb", 1000)
account2.withdraw(600)
account2.balance
```

400

```
account2.withdraw(600)
```

Insufficient funds

```
account2.balance
```

400

Inheritance

- Suppose we wanted to also create a savings account which can generate interest.
- We don't have to create a new class from scratch.
- We can create a new class that inherits the methods of the standard bank account.
- We only have to add whatever extra functionality we need.

Inheritance Example: Savings Account

```
class SavingsAccount(BankAccount):  
  
    def add_interest(self, rate):  
        self.balance *= (1 + rate / 100)  
  
account3 = SavingsAccount("Ailill", 500)  
account3.deposit(500)  
account3.balance
```

1000

```
account3.add_interest(1)  
account3.balance
```

1010.0

Checking Account

- We can also create a new class inheriting methods from another, overriding some of its methods.
- In addition, we can add additional arguments to this class.
- For example, suppose we have a checking account that charges a fee for each withdrawal.
- The fee is a new argument when creating the function.

Checking Account

```
class CheckingAccount(BankAccount):  
  
    def __init__(self, owner, balance, fee):  
        super().__init__(owner, balance)  
        self.fee = fee  
  
    def withdraw(self, amount):  
        super().withdraw(amount + self.fee)
```

Checking Account

- Here the `super()` function gives the class access to methods and properties of the parent class (`BankAccount`).
- We did not have to redefine `__init__()` completely.
- The line `super().__init__(owner, balance)` runs the `__init__()` function from the parent class and sets:

```
self.account_holder = account_holder  
self.balance = opening_balance
```

- We then only need to do the new step of setting `self.fee = fee`.

Checking Account

- In the new withdraw function, we also use `super()`.
- Here we re-use the `withdraw()` function from the parent class, but add the fee to it.
- It is equivalent to replacing it with the function:

```
def withdraw(self, amount):  
    if amount + self.fee <= self.balance:  
        self.balance -= amount + self.fee  
    else:  
        print("Insufficient funds")
```

- But notice that this is just the original `withdraw()` function with `amount` replaced with `amount + self.fee`.
- So we can just reuse our old function and just add the fee to the amount.
- This is what `super()` allows us to do.

Checking Account

Let's test it out with fee = 1:

```
account4 = CheckingAccount("Medb", 500, 1)
account4.deposit(200)
account4.balance
```

700

```
account4.withdraw(100)
account4.balance
```

599

Checking Account

- For bank accounts with the parent class BankAccount the fee is not applied:

```
account2.balance
```

400

```
account2.withdraw(100)
```

```
account2.balance
```

300

Key Terms

Now that we have seen examples of these various things, it's time to recap all the terminology we have just seen:

- A class is an object constructor. It is blueprint for creating objects. Here, `BankAccount` was a class.
- An object is a specific item created from a class. Here `account1` is an object created from the class `BankAccount`.
- An attribute stores information about an object. Here, `account holder` and `balance` are attributes of a bank account.
- A method defines an action that an object can perform. Here, `deposit` and `withdraw` were two methods we created.
- The `__init__` method sets the initial state of a new object.
- The `self` parameter refers to the object on which a method is operating.